

**SIMATS ENGINEERING
ASSESSMENT TOOL 2**

COURSE CODE: CSA51 **COURSE NAME:** Cryptography and Network Security

COURSE OUTCOME COVERED

CO3: *Examine cryptographic hash algorithms, digital signature schemes, and assess Kerberos and X.509 authentication frameworks for ensuring integrity, authentication, and non-repudiation.CO (BL4 , BL5)*

Assessment Tool Weightage: 20%

QUESTIONS: 5

TOTAL MARKS: 25

**Annexure A CASE
STUDY**

Code of Conduct:

I C SANGITA(192373059) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Sangita.c

Annexure A

1. Weak Password Hashing in Web Application

A web application stores user passwords using unsalted SHA-1 hashes. After a breach, attackers quickly crack multiple passwords using precomputed tables.

Analyze how the lack of salting and use of SHA-1 contributed to this vulnerability. Recommend a secure password hashing strategy and justify your choice.

2. Digital Signature Compromise in Financial System

A financial institution uses digital signatures for transaction approval. An attacker gains access to a user's private key and performs unauthorized transactions.

Evaluate how the compromise affects authentication and non-repudiation. Propose mitigation strategies to secure private keys and prevent such incidents.

3. Kerberos Authentication Failure in Distributed System

A distributed system using Kerberos experiences frequent authentication failures due to time synchronization issues between servers.

Analyze how time synchronization impacts Kerberos functionality. Suggest solutions to ensure reliable authentication and maintain system security.

4. Certificate Authority Breach in PKI System

A Certificate Authority (CA) is compromised, leading to the issuance of fake X.509 certificates for trusted domains.

Evaluate the impact of this breach on system trust and secure communication. Propose mechanisms to detect and mitigate such attacks in a PKI environment.

5. Integrity Verification in Cloud Data Storage

A cloud service provider uses hashing to verify file integrity, but users report data tampering incidents. Investigation reveals improper hash validation practices.

Analyze how incorrect implementation of hashing can lead to integrity failures. Recommend a robust integrity verification mechanism and justify its effectiveness.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

Answers:

Question 1: Weak password hashing in web application

Introduction

Password hashing is a security technique used to protect user passwords by converting them into a fixed-length hash value. A secure password storage system should use a strong hashing algorithm and a unique random salt for each user. In the given case, the web application stores passwords using unsalted SHA-1 hashes. After a database breach, attackers quickly crack multiple passwords using precomputed rainbow tables. This shows that the password hashing mechanism is insecure and does not provide adequate protection against modern attacks.

Case analysis

The application stores passwords as:

Password → SHA-1 → Stored Hash

Example:

- Password: admin123
- SHA-1 Hash: f865b53623b121fd34ee5426c792e5c33af8c227

Since no salt is used, two users with the same password will have the same hash value. Attackers can compare these hashes with precomputed rainbow tables and identify passwords quickly.

How lack of salting caused the vulnerability

A salt is a random value added to the password before hashing.

Without salting:

- Identical passwords produce identical hashes.
- Rainbow table attacks become possible.
- Attackers can crack many passwords simultaneously.
- Password reuse patterns become visible.

Example:

User A: password = hello123

User B: password = hello123

Both users store the same SHA-1 hash.

With salting:

User A: hello123 + salt1

User B: hello123 + salt2

The resulting hashes are completely different.

Why SHA-1 is insecure

SHA-1 is an outdated cryptographic hash function.

Main weaknesses:

- Very fast computation.
- Efficient GPU cracking.
- Collision vulnerabilities.
- Not designed for password storage.

Because SHA-1 is fast, attackers can test millions or billions of passwords per second.

Attack process

After the database breach:

1. The attacker steals the password database.
2. SHA-1 hashes are extracted.
3. Rainbow tables are used.
4. Common passwords are identified.
5. User accounts are compromised.

Impact of the attack

- User accounts become accessible.
- Sensitive personal information may be stolen.
- Password reuse may compromise other websites.
- The organization suffers financial and reputational damage.

- Customer trust is reduced.

Recommended secure password hashing strategy

The application should replace SHA-1 with Argon2id or bcrypt.

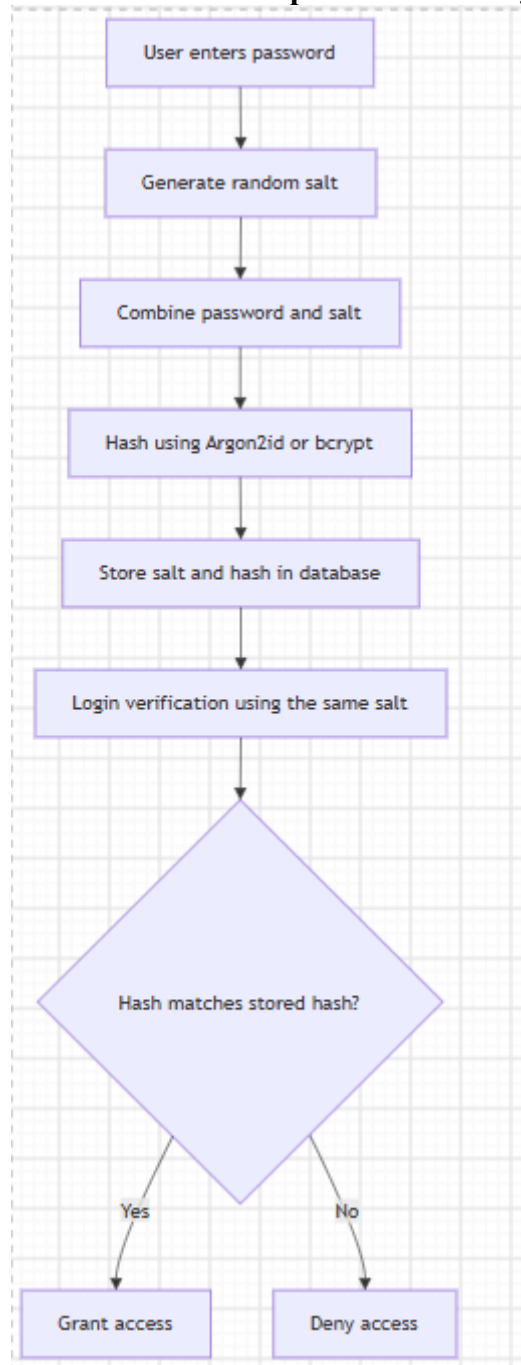
Recommended process:

Password + Random Salt → Argon2id/bcrypt → Stored Hash

Security features:

- Unique random salt for every user.
- Slow hashing algorithm.
- Configurable work factor.
- Protection against GPU attacks.
- Strong resistance to brute-force attacks.

Flowchart of secure password hashing



Python example using bcrypt

```
import bcrypt  
password = b"admin123"
```

```

salt = bcrypt.gensalt()
hashed = bcrypt.hashpw(password, salt)
print("Stored Hash:", hashed)
result = bcrypt.checkpw(password, hashed)
print("Password Verified:", result)

```

Sample output

Stored Hash: b'\$2b\$12\$Jx8FQkY0M...'

Password Verified: True

Justification

bcrypt is a better choice because:

- It automatically generates a random salt.
- It is intentionally slow.
- The work factor can be increased as hardware improves.
- It significantly increases the cost of password cracking.
- It is widely used in secure web applications.

Compared with SHA-1:

Feature	SHA-1	bcrypt
Salt support	No	Yes
Slow hashing	No	Yes
GPU resistance	Poor	Good
Password storage	Not recommended	Recommended
Work factor	No	Yes

Additional security recommendations

- Enforce strong password policies.
- Enable multi-factor authentication (MFA).
- Rate-limit login attempts.
- Monitor failed login activity.
- Periodically upgrade hashing parameters.

Conclusion

The vulnerability occurred because the application used unsalted SHA-1 hashing, allowing attackers to crack passwords rapidly using rainbow tables and brute-force attacks. The lack of salting made identical passwords easy to identify, while SHA-1's fast computation made large-scale cracking practical. The recommended solution is to use Argon2id or bcrypt with a unique random salt and an appropriate work factor. This approach effectively protects user passwords against rainbow table attacks, brute-force attacks, and GPU-based cracking, providing a secure password storage mechanism for modern web applications.

Question 2: Digital signature compromise in financial system

Introduction

A digital signature is a cryptographic technique used to verify the authenticity, integrity, and origin of electronic transactions. It uses public key cryptography, where the sender signs a transaction with a private key, and the receiver verifies it using the corresponding public key. Digital signatures provide authentication, integrity, and non-repudiation.

In the given case, an attacker gains access to a user's private key and performs unauthorized financial transactions. This compromises the security of the digital signature system.

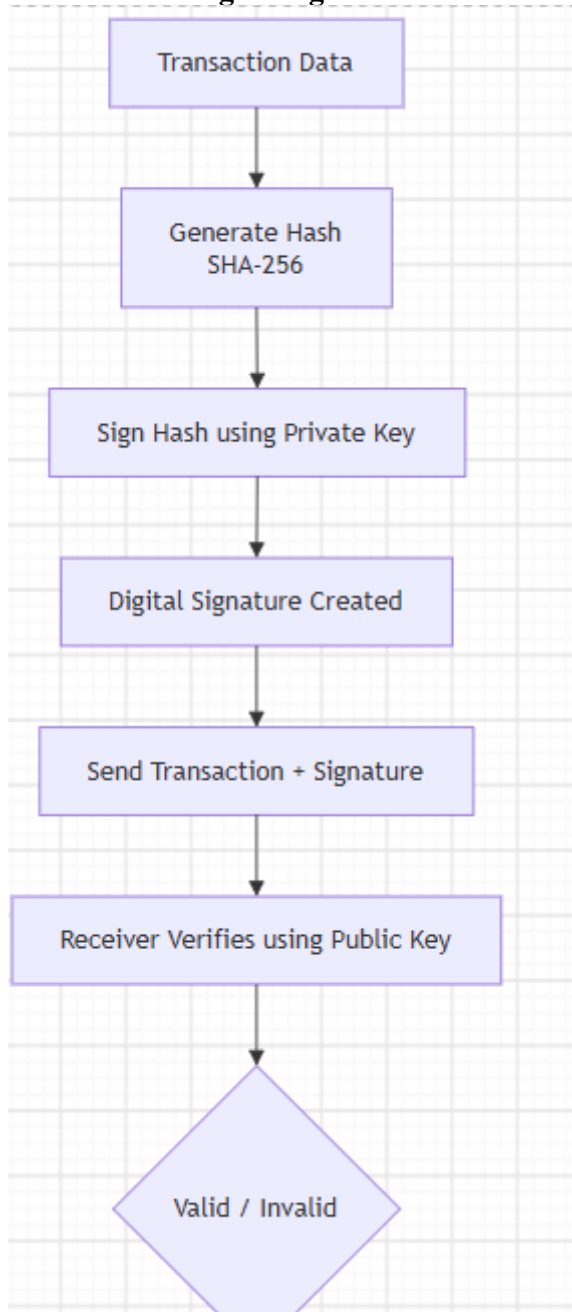
Digital signature process

The normal digital signature process is:

1. Create the transaction.

2. Generate a hash of the transaction.
3. Encrypt the hash using the sender's private key.
4. Send the transaction and signature.
5. Verify the signature using the sender's public key.

Flowchart of digital signature



Case analysis

The attacker obtains the private key of the legitimate user.

The attacker can:

- Generate valid digital signatures.
- Approve fraudulent transactions.
- Impersonate the genuine user.
- Bypass authentication checks.

Since the signature is mathematically valid, the financial institution may accept the transaction as genuine.

Effect on authentication

Authentication confirms the identity of the sender.

Before compromise:

- Only the legitimate user can sign transactions.

After compromise:

- The attacker can sign transactions.
- The system cannot distinguish between the user and the attacker.
- Identity verification fails.

Therefore, authentication is compromised.

Effect on non-repudiation

Non-repudiation means that a sender cannot deny sending a signed transaction.

After private key compromise:

- Fraudulent transactions appear legitimate.
- The genuine user may deny performing the transaction.
- The bank cannot easily prove who actually signed it.
- Legal and financial disputes may occur.

Thus, non-repudiation is weakened.

Security impact

The compromise can result in:

- Unauthorized money transfers.
- Financial fraud.
- Identity theft.
- Loss of customer confidence.
- Regulatory penalties.
- Reputation damage.

Mitigation strategies

1. Hardware Security Modules (HSMs)

Store private keys in tamper-resistant hardware.

Advantages:

- Keys cannot be extracted easily.
- Cryptographic operations occur inside the HSM.
- Physical and logical protection is provided.

2. Multi-factor authentication (MFA)

Require additional verification such as:

- OTP
- Biometric authentication
- Hardware token

Even if the private key is stolen, the attacker cannot complete the transaction without the second factor.

3. Secure key storage

Private keys should be stored:

- Encrypted.
- Password protected.
- In secure key management systems.

Keys should never be stored in plaintext.

4. Key rotation

Replace private keys periodically.

Benefits:

- Limits exposure time.
- Reduces long-term risk.
- Supports recovery after compromise.

5. Certificate revocation

When compromise is detected:

- Revoke the user's certificate.
- Publish revocation information using CRLs (Certificate Revocation Lists).

- Provide real-time status through OCSP (Online Certificate Status Protocol).

6. Transaction monitoring

Financial institutions should monitor:

- Large transactions.
- Unusual locations.
- Abnormal transaction times.
- New beneficiary accounts.

Suspicious transactions can be blocked automatically.

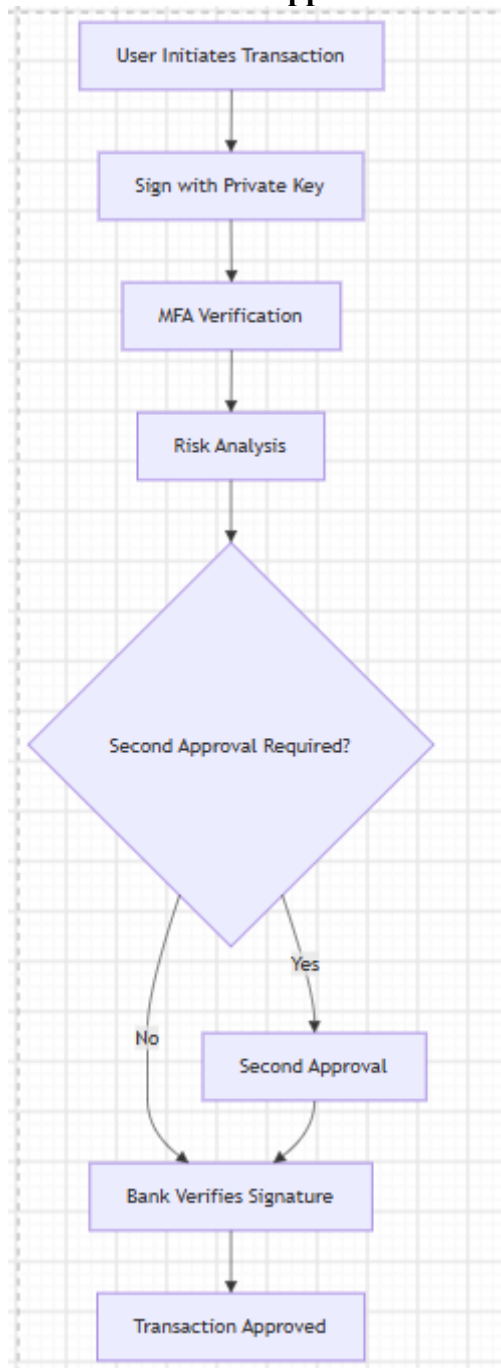
7. Dual authorization

For high-value transactions:

- User signs the transaction.
- Manager or second authorized person approves it.

This prevents a single compromised key from authorizing major transfers.

Secure transaction approval flowchart



Example of RSA digital signature

```
from Crypto.PublicKey import RSA
from Crypto.Signature import pkcs1_15
from Crypto.Hash import SHA256
key = RSA.generate(2048)
message = b"Transfer Rs.50000"
hash_value = SHA256.new(message)
signature = pkcs1_15.new(key).sign(hash_value)
print("Digital Signature Generated")
```

This example shows that the private key is used for signing, while the public key is used for verification.

Justification of the proposed solution

Security Measure	Benefit
HSM	Protects private keys from theft
MFA	Prevents unauthorized transaction approval
Key rotation	Limits damage from compromised keys
Certificate revocation	Invalidates compromised credentials
Transaction monitoring	Detects fraudulent activities
Dual authorization	Prevents single-key abuse

Conclusion

The compromise of a private key severely affects authentication and non-repudiation because attackers can generate valid digital signatures and impersonate legitimate users. The most effective protection strategy is to secure private keys using Hardware Security Modules, Multi-Factor Authentication, encrypted key storage, key rotation, certificate revocation mechanisms, transaction monitoring, and dual authorization. These measures significantly reduce the risk of unauthorized transactions and maintain trust in the financial institution's digital signature system.

Question 3: Kerberos authentication failure in distributed system (10 marks)

Introduction

Kerberos is a network authentication protocol that provides secure authentication in distributed systems using secret-key cryptography and a trusted third party called the Key Distribution Center (KDC). It authenticates users through tickets and timestamps, allowing users to access multiple services without repeatedly entering passwords.

In the given case, a distributed system experiences frequent authentication failures due to time synchronization issues between servers. Since Kerberos relies heavily on synchronized clocks, incorrect system time causes valid authentication tickets to be rejected.

Kerberos architecture

Kerberos consists of the following components:

- Client (User)
- Authentication Server (AS)
- Ticket Granting Server (TGS)
- Application Server
- Key Distribution Center (KDC)

The Authentication Server and Ticket Granting Server together form the KDC.

Kerberos authentication process

The authentication process is:

1. The client requests a Ticket Granting Ticket (TGT) from the AS.
2. The AS verifies the user and issues a TGT.
3. The client requests a service ticket from the TGS.

4. The TGS issues a service ticket.
5. The client presents the ticket to the application server.
6. The server validates the ticket and grants access.

Flowchart of Kerberos authentication

Client



Request TGT from AS



Authentication Server (AS)



Issue TGT



Request Service Ticket



Ticket Granting Server (TGS)



Issue Service Ticket



Application Server



Validate Ticket and Timestamp



Access Granted

Role of time synchronization

Kerberos tickets contain timestamps indicating when they were created.

The server checks whether the ticket time is within the permitted clock skew.

Validation condition:

$\text{Current Server Time} - \text{Ticket Time} \leq \text{Allowed Clock Skew}$

If the time difference exceeds the allowed limit, the server rejects the ticket.

Case analysis

Assume:

- Client time = 10:00 AM
- KDC time = 10:01 AM
- Server time = 10:08 AM

Time difference = 8 minutes

Since 8 minutes is greater than the allowed clock skew of 5 minutes, the application server considers the ticket invalid.

Result: Authentication failure.

How time synchronization impacts Kerberos

Time synchronization directly affects:

- Ticket validity
- Authenticator validation
- Replay attack prevention
- Mutual authentication

If clocks are not synchronized:

- Valid tickets appear expired.
- Authenticators are rejected.
- Users cannot access services.
- Authentication repeatedly fails.

Security implications

Frequent time synchronization failures can cause:

Denial of service

Legitimate users are unable to authenticate.

Reduced system availability

Distributed applications become inaccessible.

Increased administrative workload

Administrators must troubleshoot repeated authentication errors.

Potential security risk

If the clock skew limit is increased excessively, replay attacks become easier.

Replay attack protection

Kerberos prevents replay attacks using timestamps.

Without time validation:

- An attacker could capture a valid ticket.
- Replay the ticket later.
- Gain unauthorized access.

Therefore, accurate time synchronization is essential for security.

Solutions to ensure reliable authentication**1. Network Time Protocol (NTP)**

Synchronize all clients, servers, and the KDC using NTP.

Advantages:

- Automatic synchronization.
- Low time drift.
- Centralized management.

2. Redundant NTP servers

Use multiple trusted NTP servers.

Example:

- Primary NTP server
- Secondary NTP server
- Tertiary backup server

This prevents synchronization failure if one server becomes unavailable.

3. Continuous clock monitoring

Monitor systems for:

- Clock drift.
- Synchronization failure.
- Large time offsets.

Alerts should be generated when time differences exceed acceptable limits.

4. Secure NTP

Protect NTP communications using:

- Authentication.
- Trusted time sources.
- Access controls.

This prevents attackers from manipulating system time.

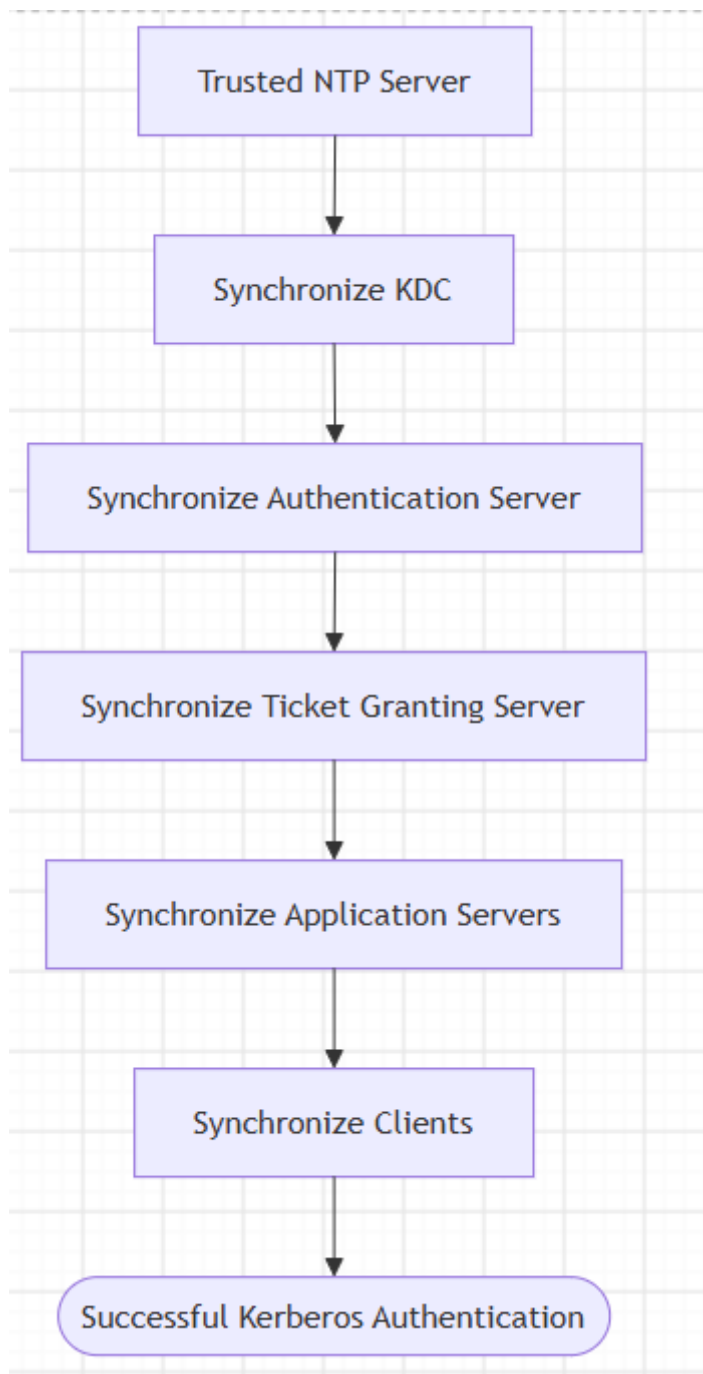
5. Kerberos configuration

Maintain:

- Reasonable clock skew (5 minutes).
- Ticket lifetime limits.
- Ticket renewal policies.

Avoid excessively increasing clock skew because it weakens replay protection.

NTP synchronization flowchart



Example scenario

Before synchronization:

- Authentication failures: 300 per day.
- Average clock drift: 8 minutes.

After NTP implementation:

- Authentication failures: 5 per day.
- Clock drift: less than 30 seconds.

This demonstrates the importance of synchronized clocks.

Advantages of proper time synchronization

Benefit	Description
Reliable authentication	Tickets are accepted correctly
Replay protection	Old tickets are rejected
Mutual authentication	Client and server verify each other
Service availability	Users access services successfully
Security maintenance	Timestamp validation remains effective

Conclusion

Kerberos authentication depends heavily on accurate time synchronization because tickets and authenticators contain timestamps used for validation and replay attack prevention. In the given case, clock differences between clients, servers, and the KDC caused legitimate tickets to be rejected, resulting in authentication failures. The most effective solution is to implement secure Network Time Protocol (NTP), redundant time servers, continuous clock monitoring, and proper Kerberos clock skew configuration. These measures ensure reliable authentication, maintain replay attack protection, and preserve the overall security of the distributed system.

Question 4: Certificate authority breach in PKI system (10 marks)

Introduction

A Public Key Infrastructure (PKI) is a framework that manages digital certificates and public-key encryption. An X.509 certificate is issued by a Certificate Authority (CA) to verify the identity of websites, servers, and users. The CA acts as the trusted third party in the PKI system.

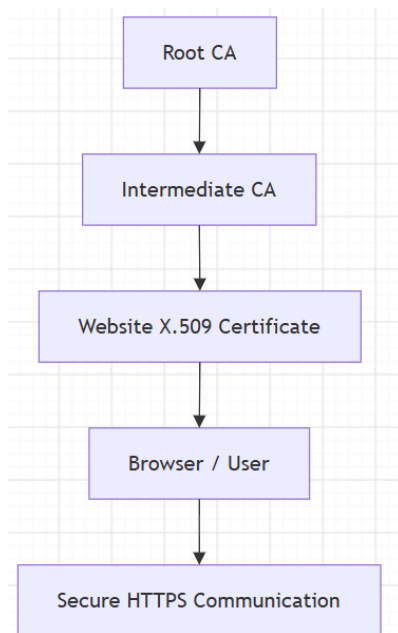
In the given case, a Certificate Authority is compromised, allowing attackers to issue fake X.509 certificates for trusted domains. This is one of the most serious security failures in a PKI environment because the CA forms the foundation of trust for secure communication.

PKI architecture

A PKI system consists of:

- Root Certificate Authority
- Intermediate Certificate Authority
- Registration Authority
- End users
- Web servers
- Browsers and applications

PKI trust chain



Browsers trust certificates issued by trusted CAs. If the CA is compromised, this trust chain becomes unreliable.

Case analysis

After compromising the CA, attackers can generate valid X.509 certificates for domains such as:

- Banking websites
- Government portals
- E-commerce websites
- Email servers
- Corporate websites

Because these certificates are signed by a trusted CA, browsers may accept them as legitimate.

Impact on system trust

The compromise affects the entire trust model of PKI.

Impersonation attacks

Attackers can create certificates for legitimate domains and impersonate trusted websites.

Example:

- Real website: <https://bank.com>
- Fake website: <https://bank.com>

Users cannot easily distinguish between them.

Man-in-the-middle (MITM) attacks

Attackers intercept communication between users and websites.

Attack process:

User

↓

Attacker with Fake Certificate

↓

Real Website

The attacker can:

- Read encrypted traffic.
- Modify data.
- Steal passwords.
- Capture banking information.

Loss of confidentiality

Sensitive information may be exposed, including:

- Login credentials.
- Financial data.

- Personal information.
- Business communications.

Loss of integrity

Attackers may modify transmitted data.

Loss of authentication

Users may unknowingly communicate with malicious servers.

Damage to PKI trust

The compromised CA can no longer be fully trusted.

Organizations may need to replace:

- Certificates.
- Keys.
- Trust relationships.

Detection mechanisms

Certificate Transparency (CT)

CT maintains public logs of all issued certificates.

Benefits:

- Detect unauthorized certificates.
- Monitor domain certificates.
- Identify suspicious issuance.

Certificate monitoring

Organizations continuously monitor CT logs for their domains.

If an unexpected certificate appears, they can respond immediately.

Online Certificate Status Protocol (OCSP)

Browsers check certificate validity in real time.

Possible responses:

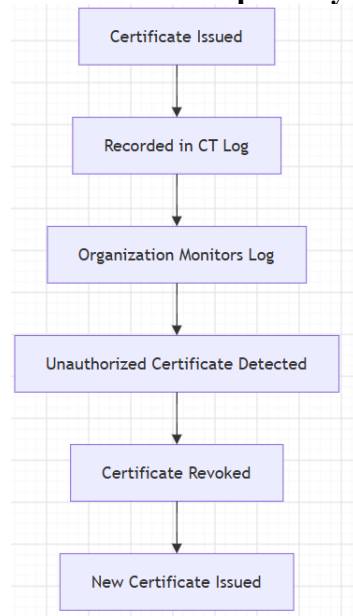
- Good
- Revoked
- Unknown

Certificate Revocation Lists (CRLs)

CRLs contain serial numbers of revoked certificates.

Browsers reject certificates appearing in the revocation list.

Certificate transparency flowchart



Mitigation strategies

Immediate certificate revocation

Revoke all fraudulent certificates immediately.

Reissue certificates

Generate:

- New key pairs.
- New certificate signing requests.
- New trusted certificates.

Hardware Security Modules (HSMs)

Store CA private keys inside tamper-resistant hardware.

Advantages:

- Prevent key extraction.
- Secure signing operations.
- Physical protection.

Multi-factor authentication for CA administrators

Require:

- Password
- Hardware token
- Biometric authentication

This reduces insider threats.

Certificate pinning

Applications accept only specific trusted certificates.

Even if a fraudulent certificate exists, the application rejects it.

Short-lived certificates

Reduce certificate validity periods.

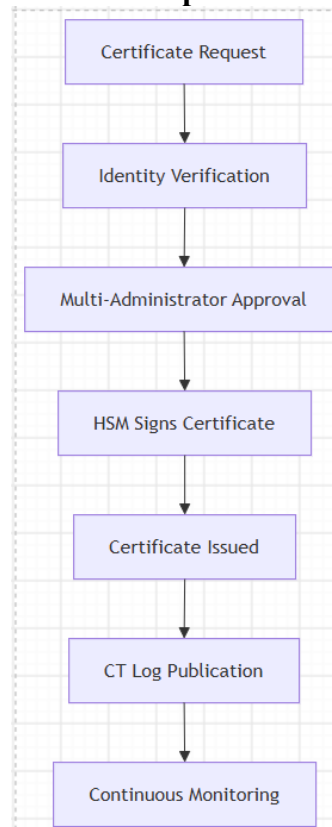
Benefits:

- Faster replacement.
- Reduced exposure time.
- Improved security.

Multi-person approval

Require multiple administrators before issuing high-value certificates.

Secure CA operation flowchart



Example of certificate verification

A browser receives:

- Server certificate
- Intermediate CA certificate
- Root CA certificate

The browser verifies:

1. Digital signature.
2. Certificate validity period.
3. Domain name.
4. Revocation status.
5. Trust chain.

If the CA is compromised, signature verification may succeed even for malicious certificates.

Advantages of the proposed mechanisms

Security Mechanism	Benefit
Certificate Transparency	Detects unauthorized certificates
OCSP	Real-time revocation checking
CRLs	Revoked certificate identification
HSM	Protects CA private keys
Certificate pinning	Prevents acceptance of fake certificates
Multi-factor authentication	Secures CA administration
Short-lived certificates	Reduces attack window

Conclusion

A Certificate Authority breach severely damages the trust foundation of a PKI system. Attackers can issue fake X.509 certificates, perform man-in-the-middle attacks, impersonate trusted domains, and compromise secure HTTPS communication. The most effective defense includes Certificate Transparency, OCSP, CRLs, Hardware Security Modules, certificate pinning, multi-factor authentication, and strong operational security controls. These mechanisms help detect fraudulent certificates, protect CA private keys, and restore trust in the PKI infrastructure.

Question 5: Integrity verification in cloud data storage (10 marks)

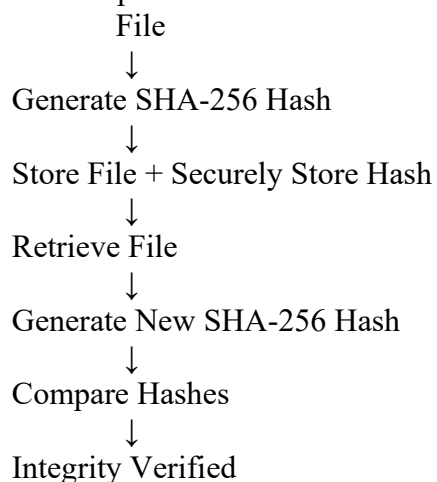
Introduction

Cloud storage allows users to store files on remote servers and access them from anywhere through the internet. Since the data is stored on third-party infrastructure, data integrity becomes one of the most important security requirements. Integrity means that the data remains accurate, complete, and unmodified during storage and transmission.

Understanding hash-based integrity verification

A cryptographic hash function produces a unique fingerprint of the data.

Normal process:



Any modification to the file changes the hash value.

Example:

Original File: Report.pdf

SHA-256:

A3F9B2C4D5E6F789...

If a single character changes, the hash becomes completely different.

Case analysis

The cloud service provider used hashing for integrity verification, but the implementation was incorrect.

Possible implementation errors include:

- Weak hash algorithms.
- Improper hash comparison.
- Failure to verify hashes after retrieval.
- Insecure storage of hash values.
- Allowing modification of both the file and the hash.
- Ignoring verification failures.

Because of these mistakes, tampered files were accepted as valid.

How incorrect hashing leads to integrity failures

Use of weak hash algorithms

Algorithms such as MD5 and SHA-1 are no longer considered secure.

Problems:

- Collision attacks.
- Faster brute-force attacks.
- Reduced cryptographic strength.

Attackers may create different files with the same hash value.

Insecure storage of hashes

Suppose the system stores:

- File
- Hash value

If an attacker gains access to storage, they can modify both.

Original file



Original SHA-256 hash



Attacker changes file



Attacker replaces stored hash



System compares modified file with modified hash



Integrity check falsely succeeds

Missing validation during retrieval

If the provider verifies the hash only during upload and not during download, later modifications remain undetected.

Incorrect hash comparison

Improper implementation may use:

- Partial hash comparison.
- Truncated hashes.
- Case-insensitive comparison.
- Timing-vulnerable comparison.

These mistakes may incorrectly mark tampered data as authentic.

Impact of integrity failure

Improper integrity verification can cause:

Data tampering

Documents may be modified without detection.

Malware insertion

Attackers may replace legitimate files with malicious software.

Financial fraud

Invoices, transaction records, and contracts may be altered.

Loss of customer trust

Users lose confidence in the cloud provider.

Regulatory violations

Organizations may fail compliance requirements such as GDPR, HIPAA, or ISO 27001.

Business disruption

Corrupted data may affect operations and decision-making.

Recommended robust integrity verification mechanism

A stronger solution is HMAC-SHA-256 (Hash-based Message Authentication Code).

HMAC combines:

- SHA-256 hash function.
- Secret cryptographic key.

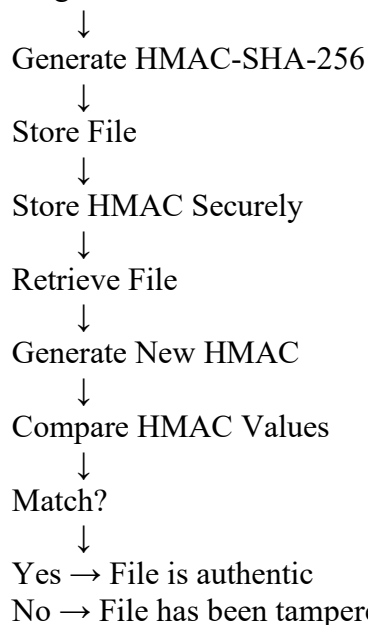
Formula:

$\text{HMAC} = \text{SHA-256}(\text{Key} + \text{Message})$

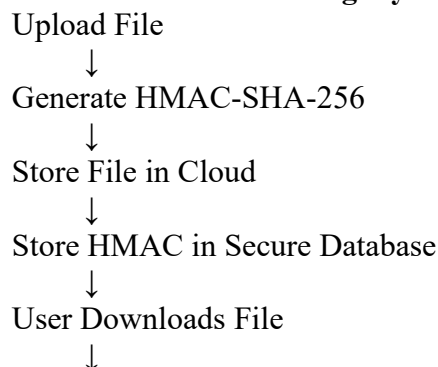
Without the secret key, attackers cannot generate a valid HMAC for modified data.

HMAC integrity verification process

Original File



Flowchart of secure integrity verification



Recalculate HMAC



Compare with Stored HMAC



Match?



Yes → Provide File

No → Reject File and Generate Alert

Python example using HMAC-SHA-256

```
import hmac
import hashlib
# Secret key
key = b"cloud_secret_key"
# File data
data = b"Cloud storage file contents"
# Generate HMAC
hmac_value = hmac.new(key, data, hashlib.sha256)
print("HMAC:", hmac_value.hexdigest())
```

Sample output:

HMAC:

9d5f6b8c4a1f3d2e8b9c7f6a5d4c3b2a1f9e8d7c6b5a4f3e2d1c0b9a8f7e6d5

During verification, the HMAC is recalculated and compared with the stored value.

Merkle tree for large cloud storage

For large files and distributed cloud systems, Merkle trees provide efficient integrity verification.

Structure:

Hash(Block1)

Hash(Block2)



Hash12

Hash(Block3)

Hash(Block4)



Hash34



Root Hash

Advantages:

- Efficient verification.
- Detects which block was modified.
- Suitable for distributed storage systems.
- Reduces verification overhead.

Only the affected block needs verification.

Additional security mechanisms

Digital signatures

The file owner signs the hash using a private key.

Provides:

- Integrity.
- Authentication.
- Non-repudiation.

Immutable storage

Files cannot be modified after upload.

Useful for:

- Legal records.

- Audit logs.
- Financial data.

Versioning

Maintain previous versions of files.

Benefits:

- Recovery after tampering.
- Accidental deletion protection.
- Historical auditing.

Audit logging

Record:

- Upload events.
- Download events.
- Hash verification.
- Integrity failures.
- Administrative actions.

Logs help detect insider attacks.

Third-party auditing

An independent auditor verifies data integrity periodically without exposing the file contents.

Comparison of integrity verification methods

Method	Integrity Authentication		Security Level
MD5	Weak	No	Low
SHA-1	Weak	No	Low
SHA-256	Strong	No	Good
HMAC-SHA-256	Strong	Yes	Very High
Digital Signature	Strong	Yes	Very High

Advantages of the proposed solution

HMAC-SHA-256 provides:

- Strong tamper detection.
- Protection against hash forgery.
- Authentication of the data source.
- Resistance to collision attacks.
- Efficient verification.
- Wide industry acceptance.

Merkle trees improve scalability for large cloud storage systems.

Justification

The proposed mechanism is effective because:

- Attackers cannot generate valid HMAC values without the secret key.
- SHA-256 provides strong cryptographic protection.
- Secure hash storage prevents unauthorized replacement.
- Verification during retrieval detects modifications immediately.
- Merkle trees enable efficient verification of large datasets.
- Audit logs support forensic investigation.

Therefore, the mechanism provides integrity, authenticity, and reliable tamper detection.

Conclusion

In the given case, the cloud provider's improper hash validation practices allowed data tampering incidents to occur. Weak algorithms, insecure hash storage, and incorrect verification procedures caused integrity failures. A robust solution is to implement HMAC-SHA-256 with secure key management, mandatory verification during file retrieval, Merkle tree verification for large files, immutable storage, versioning, and audit logging. These mechanisms provide strong protection against unauthorized modification and ensure reliable integrity verification in cloud data storage environments.